Dev Days

Kathmandu, Nepal

Build. Ship. Repeat. With GitHub Copilot.

Puru Tuladhar

Senior Cloud Infrastructure Engineer
CoverGo

"What Devs Should Know About DevOps: From Code to Production"

Sept 5
12:30 - 4:00 PM

Leapfrog Connect
Radhe Marg, Dillibazar

Organized by

Supported by

Venue Partner

Creative Partner

Dev Days / Kathmandu, Nepal · Sept 5, 2026

What Developers Should Know About DevOps

from Code → Production

Puru Tuladhar

Senior Cloud Infrastructure Engineer, CoverGo · CNCF Kubestronaut

10+ Years in DevOps

purutuladhar.com

The road from your code to production

Based on `roadmap.sh/devops` · twelve stops, and you already own the first one.

Your machine

1. Programming language
2. Linux
3. Bash & the terminal
4. Code editor

Shipping the code

5. Git
6. Containers
7. Networking & protocols

Where it runs

8. Cloud
9. Infrastructure as code
12. Container orchestration

Keeping it alive

10. Logs, monitoring, observability
11. Secrets management

`roadmap.sh/devops` This talk follows the DevOps roadmap — the same map, from a developer's side of it.

Before we start — hands up

No wrong answers, and no one is writing this down.

Who's in the room

1. How many of you are developers?
2. How many are DevOps engineers?
3. How many are full-stack?
4. How many are switching to DevOps?

And be honest

5. Who has pushed straight to `main` in production?
6. Who has said "but it works on my machine"?
7. Who has been woken up by something they shipped?
8. Who has deployed by clicking around a cloud console?
9. Who wants to stop depending on someone else to ship?

`warm-up` Every hand in this room has already met at least one stop on this road.

You already have this one

A language you can build something real in. Most of this room is here already — that's the whole starting requirement.

This is the only stop on the road that's about **writing** code. Every stop after it is about what happens to that code once it's written.

Pick one, go deep

- Python
- Go
- JavaScript / Node.js
- Ruby
- Rust
- Java / C#

The language matters far less than what you can ship with it.

Where we are

✓ Programming language

Your machine

- ✓ Programming language
- 2 Linux
- 3 Bash & the terminal
- 4 Code editor

Shipping the code

- 5 Git
- 6 Containers
- 7 Networking & protocols

Where it runs

- 8 Cloud
- 9 Infrastructure as code
- 12 Container orchestration

Keeping it alive

- 10 Logs, monitoring, observability
- 11 Secrets management

roadmap.sh/devops

02 Your machine · Linux

Production runs on Linux

Your app may be written anywhere, but it almost certainly runs on a Linux box in production. Being a stranger to that box is what makes deployment feel like someone else's job.

Start with **Ubuntu** on the server — it's what you'll meet most often, in every cloud, in every base image.

What to actually learn

- Files, paths, permissions
- Processes and what's eating the CPU
- systemd: start, stop, why it died
- journalctl and /var/log
- Packages, users, SSH

Enough to log in at 2am and answer "what is this machine doing?"

stop 02 of 12 Server-side is Ubuntu. Desktop-side, there's something worth your evening.

02 Your machine · Linux on the desktop

Omarchy

Learning Linux goes faster when you live in it. Omarchy is a ready-made Arch desktop — opinionated, fast, and set up before you get there.

And in the agentic era, the desktop matters again: you want a machine where trying a new coding agent is one command, not an afternoon.

agents, pre-installed

`copilot`	GitHub Copilot CLI	`claude`	Claude Code
`codex`	OpenAI Codex	`opencode`	OpenCode
`agy`	Google Antigravity	`crush`	Crush

Omarchy Kathmandu

Join the group on LinkedIn

See how it feels

`omarchy.nkz.md`

`community` We ran the first Omarchy Kathmandu meetup yesterday. This is the group that came out of it.

Where we are

✓ Linux

Your machine

- ✓ Programming language
- ✓ Linux
- 3 Bash & the terminal
- 4 Code editor

Shipping the code

- 5 Git
- 6 Containers
- 7 Networking & protocols

Where it runs

- 8 Cloud
- 9 Infrastructure as code
- 12 Container orchestration

Keeping it alive

- 10 Logs, monitoring, observability
- 11 Secrets management

roadmap.sh/devops

Bash is everywhere

Every build pipeline you will ever touch runs shell somewhere. The steps in your CI file are Bash. The container entrypoint is Bash. The fix at 2am is Bash.

Frameworks come and go every few years. The shell you learn today still works in **ten years** — that's why it's the best long-term investment on this road.

.github/workflows/build.yml

```
steps:
  - run: |
      # this is Bash
      set -euo pipefail
      ./scripts/build.sh
      docker build -t app:$TAG .
```

Learn enough to read it, debug it, and write it without guessing.

stop 03 of 12 Be comfortable in the terminal and half of DevOps stops being intimidating.

Where we are

✓ Bash & the terminal

Your machine

- ✓ Programming language
- ✓ Linux
- ✓ Bash & the terminal
- 4 Code editor

Shipping the code

- 5 Git
- 6 Containers
- 7 Networking & protocols

Where it runs

- 8 Cloud
- 9 Infrastructure as code
- 12 Container orchestration

Keeping it alive

- 10 Logs, monitoring, observability
- 11 Secrets management

roadmap.sh/devops

Use whatever makes you fast

This is the one stop on the road with no right answer. VS Code, Cursor, Neovim, Emacs — the best editor is the one you stop thinking about while you work.

The only question worth asking now: how well does it let you **work with an assistant**? That's where the difference between editors is actually showing up.

Whatever you pick, invest in

- Keyboard shortcuts you use daily
- Debugger, not print statements
- Integrated terminal and Git
- Copilot or your agent of choice

No one has ever been promoted for their editor. Plenty have been slowed down by never learning one properly.

Where we are

✓ Code editor

Your machine

- ✓ Programming language
- ✓ Linux
- ✓ Bash & the terminal
- ✓ Code editor

Shipping the code

- 5 **Git**
- 6 Containers
- 7 Networking & protocols

Where it runs

- 8 Cloud
- 9 Infrastructure as code
- 12 Container orchestration

Keeping it alive

- 10 Logs, monitoring, observability
- 11 Secrets management

roadmap.sh/devops

05 Shipping the code · Git

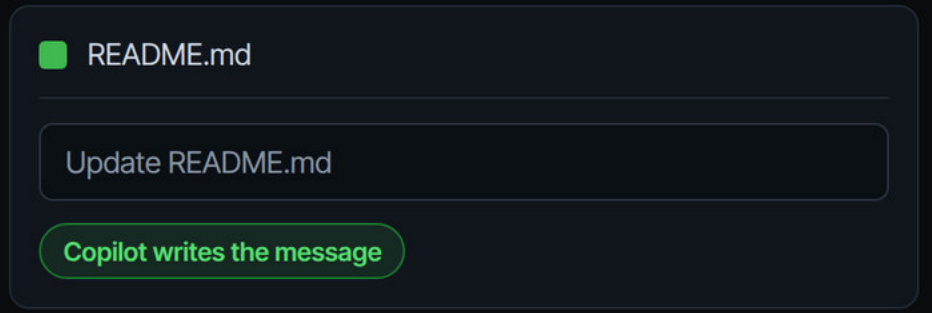
Git is the one you touch every day

GitHub, GitLab, Bitbucket — they all start from the same local Git client. Learn Git properly and the platform stops mattering.

Your commits and your pull request are what the rest of the team actually reads. A PR that's easy to review gets merged fast; a 40-file PR with one line of description sits for three days.

Worth getting right

- Commit messages that explain *why*
- Small, scoped, reviewable PRs
- Tags for versioning your releases
- Branching your team agrees on



stop 05 of 12 Review quality is part of the SDLC, not an afterthought at the end of it.

Where we are

✓ Git

Your machine

- ✓ Programming language
- ✓ Linux
- ✓ Bash & the terminal
- ✓ Code editor

Shipping the code

- ✓ Git
- 6 **Containers**
- 7 Networking & protocols

Where it runs

- 8 Cloud
- 9 Infrastructure as code
- 12 Container orchestration

Keeping it alive

- 10 Logs, monitoring, observability
- 11 Secrets management

roadmap.sh/devops

Containers are how code travels

A container is your app plus everything it needs to run, packaged once and identical everywhere. It's what ended “works on my machine” and what makes microservices deployable at all.

Anyone can write a Dockerfile that works. The skill worth having is writing one that's **small and safe** — because that image is what actually runs in production.

Dockerfile

```
# build stage
FROM node:22 AS build
RUN npm ci && npm run build

# ship only what runs
FROM node:22-slim
COPY --from=build /app/dist ./
USER node
CMD ["node","server.js"]
```

Multi-stage build, slim base, non-root user. Smaller image, fewer vulnerabilities, faster deploys.

Where we are

✓ Containers

Your machine

- ✓ Programming language
- ✓ Linux
- ✓ Bash & the terminal
- ✓ Code editor

Shipping the code

- ✓ Git
- ✓ Containers
- 7 Networking & protocols

Where it runs

- 8 Cloud
- 9 Infrastructure as code
- 12 Container orchestration

Keeping it alive

- 10 Logs, monitoring, observability
- 11 Secrets management

roadmap.sh/devops

Know how the request reaches you

Frameworks change every year. The protocols underneath have barely moved in twenty. Learn them once and you can debug any stack.

Most production mysteries — the timeout, the certificate error, the "works locally" failure — are **network problems wearing an application costume**.

One request, end to end

DNS	name becomes an address
TCP	connection opens
TLS	certificate check, encryption
HTTP	load balancer routes it
:8080	your app finally sees it

Plus SSH, because that's how you get to the box.

Where we are

✓ Networking & protocols

Your machine

- ✓ Programming language
- ✓ Linux
- ✓ Bash & the terminal
- ✓ Code editor

Shipping the code

- ✓ Git
- ✓ Containers
- ✓ Networking & protocols

Where it runs

- 8 Cloud
- 9 Infrastructure as code
- 12 Container orchestration

Keeping it alive

- 10 Logs, monitoring, observability
- 11 Secrets management

roadmap.sh/devops

08 Where it runs · Cloud

Cloud made infrastructure available to everyone

A team of three in Kathmandu can run on the same infrastructure as a bank. No data centre, no procurement — you spend your attention on the software instead.

Learn **one** provider properly rather than three badly. The concepts carry over; the console never does.

Pick one

AWS | Azure | Google Cloud

Then get certified on it

An associate-level certificate is a few weekends of study, and on a CV in this market it is a genuine differentiator.

Where we are

✓ Cloud

Your machine

- ✓ Programming language
- ✓ Linux
- ✓ Bash & the terminal
- ✓ Code editor

Shipping the code

- ✓ Git
- ✓ Containers
- ✓ Networking & protocols

Where it runs

- ✓ Cloud
- 9 Infrastructure as code
- 12 Container orchestration

Keeping it alive

- 10 Logs, monitoring, observability
- 11 Secrets management

roadmap.sh/devops

Stop clicking. Start committing.

Clicking through a console is fine to learn. In production it's a liability: nobody knows who changed what, and nobody can rebuild it.

Infrastructure as code makes your servers reviewable, versioned and repeatable — the same way your application already is. **You already know how to work this way.**

main.tf

```
resource "aws_s3_bucket" "app" {
  bucket = "devdays-ktm-app"
  tags = {
    Owner = "platform"
  }
}
```

Terraform | Pulumi | AWS CDK

Terraform is the default. CDK and Pulumi let you write it in a language you already use.

Where we are

✓ Infrastructure as code

Your machine

- ✓ Programming language
- ✓ Linux
- ✓ Bash & the terminal
- ✓ Code editor

Shipping the code

- ✓ Git
- ✓ Containers
- ✓ Networking & protocols

Where it runs

- ✓ Cloud
- ✓ Infrastructure as code
- 12 Container orchestration

Keeping it alive

- 10 **Logs, monitoring, observability**
- 11 Secrets management

roadmap.sh/devops

Software breaks. Then what?

When it breaks in production you need your logs in one place, searchable, without SSH-ing into anything. That's centralised logging.

But you shouldn't find out from a customer. Monitoring and alerting is what tells you first — for the infrastructure *and* for the things only your application knows.

The stack most teams land on

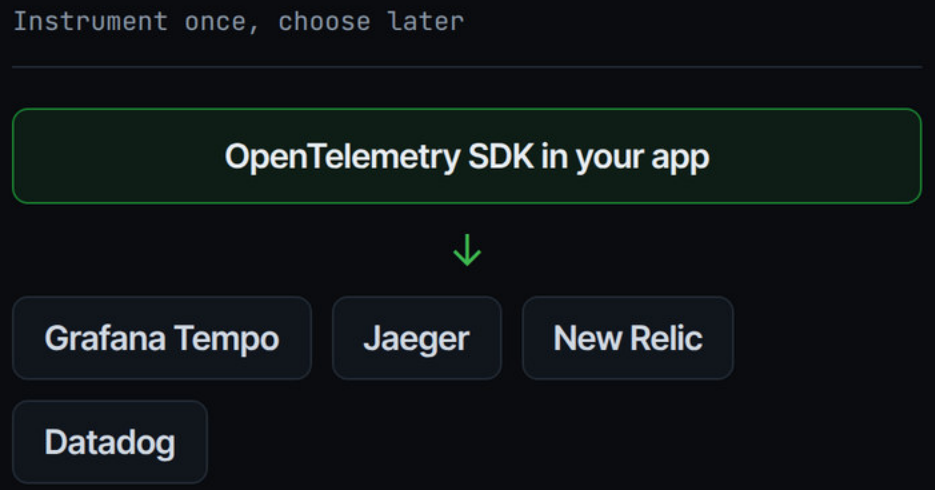
Grafana	dashboards and alerts
Loki	logs, searchable
Prometheus	metrics over time

Alert on what a user would notice. Everything else is a dashboard, not a page at 3am.

Twelve services, one slow request

Once your system is a dozen services, “the checkout is slow” isn't answerable from logs alone. You need to follow one request across every hop it took.

That's tracing, and **OpenTelemetry** is now the standard way to emit it — instrument once, send it wherever you like.



No vendor lock-in at the point where you can least afford it.

Where we are

✓ Logs, monitoring & observability

Your machine

- ✓ Programming language
- ✓ Linux
- ✓ Bash & the terminal
- ✓ Code editor

Shipping the code

- ✓ Git
- ✓ Containers
- ✓ Networking & protocols

Where it runs

- ✓ Cloud
- ✓ Infrastructure as code
- 12 Container orchestration

Keeping it alive

- ✓ Logs, monitoring, observability
- 11 **Secrets management**

roadmap.sh/devops

Never commit the secret

Not in Git, not baked into the image, not hardcoded in the source. Git history is forever, and a pushed secret is compromised the moment it lands.

Every cloud gives you a secret manager. Read from it at startup, inject as environment variables, and **never print it to logs** — a logged secret is a production security incident and a rotation you'll do at midnight.

Where secrets should live

- AWS Secrets Manager, Azure Key Vault, GCP Secret Manager
- HashiCorp Vault
- Sealed Secrets or External Secrets on Kubernetes

Handling secrets well is the cheapest security reputation a developer can build.

stop 11 of 12 Assume everything you log will be read by someone else.

Where we are

✓ Secrets management

Your machine

- ✓ Programming language
- ✓ Linux
- ✓ Bash & the terminal
- ✓ Code editor

Shipping the code

- ✓ Git
- ✓ Containers
- ✓ Networking & protocols

Where it runs

- ✓ Cloud
- ✓ Infrastructure as code
- 12 Container orchestration

Keeping it alive

- ✓ Logs, monitoring, observability
- ✓ Secrets management

roadmap.sh/devops

One container is easy. Two hundred is Kubernetes.

Kubernetes is where almost all container workloads run now. As a developer you don't need to operate a cluster — you need to deploy onto one confidently.

Two things earn their keep immediately: **scaling under load**, and **rolling updates with zero downtime**. Both are a few lines of YAML.

What to learn as a dev

- Pods, Deployments, Services, Ingress
- ConfigMaps and Secrets
- Readiness and liveness probes
- Rolling updates and rollbacks

Start with CKAD

It's the developer-facing certification. Take the course even if you never sit the exam — the syllabus is exactly this list.

stop 12 of 12 In the cloud you'll use managed Kubernetes: EKS, AKS or GKE.

Where this actually takes you

The same twelve stops, seen as a career rather than a syllabus.

Becoming a dev
Get the machine under your control.
Programming language · Linux · Bash · Code editor

Dev → Senior
Know how the pieces fit together, not just your own.
Git · Containers · Networking · Cloud

Senior → Full-stack
Own it end to end — you deploy it, you run it.
Infrastructure as code · Observability · Secrets · Kubernetes

Full-stack → Lead
Architecture on top, and a team you bring with you.
System design · Trade-offs · Reviews · Mentoring

Lead → Product
Drive the business forward with what the stack makes possible.
Business value · Cost · Innovation · Roadmap

`takeaway` Everybody moves at their own pace.

Thank you

Questions?

Puru Tuladhar

purutuladhar.com

Dev Days · Kathmandu, Nepal · Sept 5, 2026

Let's connect

linkedin.com/in/ptuladhar3